PLANTILLA Y ENTORNO

```cpp
#include<bits/stdc++.h>
using namespace std;
// --- --- --- --- Type Aliases --- --- --- ---
typedef int64_t ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vll;
typedef vector<vi> vvi;
typedef vector<vll> vvll;
typedef vector<pii> vpii;
typedef vector<pll> vpll;
// --- --- --- --- Short Macros --- --- --- ---
#define fi first
#define se second
#define pb push_back
#define all(x) (x).begin(), (x).end()
#define sz(x) (int)(x).size()
#define rep(i,a,b) for( int i=a; i< (b); ++i )
const int MOD = 1e9 + 7;
// --- --- --- --- LOGGER SETUP --- --- --- ---
template<typename C, typename T = typename
  ↪ enable_if<!is_same<C, string>::value, typename
  ↪ C::value_type>::type>
ostream& operator<<(ostream &os, const C &v);
// Overload for std::pair
template<typename A, typename B>
ostream& operator<<(ostream &os, const pair<A, B> &p) {
    return os << '(' << p.fi << ", " << p.se << ')';
}
// Overload for Containers (excluding std::string) SFINAE
template<typename C, typename T>
ostream& operator<<(ostream &os, const C &v) {
    string sep;
    for(const T &x : v) os << sep << x, sep = " ";
    return os;
}
// colores
constexpr const char *RST="\033[0m", *GREEN="\033[1;32m",
*YELLOW="\033[1;33m",*CYAN="\033[1;36m",*RED="\033[1;31m";
// Logger Function
template<typename ...Args>
void logger(const char* col, string vars, Args&&... values){
    cerr << col << '[' << vars << "] =" << RST;
    ((cerr << " [" << values << ']'), ...);
    cerr << '\n';
}
// --- --- --- --- DEBUGGER SETUP --- --- --- ---
#ifdef LOCAL
  #define DEBUG   if (true)
  #define SALTO   (cerr << '\n')
  #define LOG(...)   logger("", #__VA_ARGS__, __VA_ARGS__)
  #define LOGC(c, ...) logger(c, #__VA_ARGS__, __VA_ARGS__)
  #define COL(c, ...) (cerr << c << __VA_ARGS__ << RST << '\n')
#else
    #define DEBUG   if (false)]
    #define SALTO
    #define LOG(...)
    #define LOGC(...)
    #define COL(...)
#endif
// // // // // // // // // // // // // // // // // // // // // // // // //
int main(){
    #ifdef LOCAL
        ifstream cin("in.in");
    #else
        ios_base::sync_with_stdio(0); cin.tie(0); cout.tie(0);
    #endif
}
```

En el archivo `~/.bashrc` (o `~/.zshrc` para macOS) escribir:

```
alias gc='g++ -std=c++17 -O3 -Wall -Wextra -DLOCAL'
alias g='g++ -std=c++17 -O3 -Wall -Wextra'
```

y refrescar `source  /.zshrc`

▶ Memoria disponible

```
-Wl,-stack_size,0x8000000 // 128 MB  // 0x10000000 // 256 MB
-Wl,-stack_size,0x20000000 // 512 MB // 0x40000000 // 1 GB
ulimit -s                 ulimit -s 524288 // 512MB
ulimit -s unlimited       ulimit -s 1048576 // 1gb
```

INFORMACION ADICIONAL

▶ Constantes y más

- ***Límites de tipos***

`INT_MAX`	$2{,}147{,}483{,}647 = 2^{31}-1 \approx 2.1 \times 10^9$
`INT_MIN`	$-2^{31} = -2{,}147{,}483{,}648$
`UINT_MAX`	$4{,}294{,}967{,}295 = 2^{32}-1 \approx 4.29 \times 10^9$
`LLONG_MAX`	$9{,}223{,}372{,}036{,}854{,}775{,}807 = 2^{63}-1 \approx 9.22 \times 10^{18}$
`LLONG_MIN`	-2^{63}
`ULLONG_MAX`	$2^{64}-1 = 18{,}446{,}744{,}073{,}709{,}551{,}615 \approx 1.84 \times 10^{19}$
`double`	enteros exactos hasta $2^{53} \approx 9.007 \times 10^{15}$

- ***Valores de INF recomendados***

`int`: `INF` $= 10^9$ (o `0x3f3f3f3f` $\approx 1.06 \times 10^9$, permite `INF` + `INF` sin overflow).
`long long`: `INF` $= 10^{18}$ (o `4e18`; ojo con sumar dos INF).

- ***Potencias de 2***

2^{10}	1,024	2^{19}	524,288	2^{37}	$\approx 1.37 \times 10^{11}$
2^{11}	2,048	2^{20}	1,048,576	2^{40}	$\approx 1.10 \times 10^{12}$
2^{12}	4,096	2^{22}	4,194,304	2^{45}	$\approx 3.52 \times 10^{13}$
2^{13}	8,192	2^{25}	33,554,432	2^{50}	$\approx 1.13 \times 10^{15}$
2^{14}	16,384	2^{27}	134,217,728	2^{60}	$\approx 1.15 \times 10^{18}$
2^{15}	32,768	2^{30}	$\approx 1.07 \times 10^9$	2^{63}	$\approx 9.22 \times 10^{18}$
2^{16}	65,536	2^{31}	$\approx 2.15 \times 10^9$	2^{64}	$\approx 1.84 \times 10^{19}$
2^{17}	131,072	2^{32}	$\approx 4.29 \times 10^9$	2^{127}	$\approx 1.70 \times 10^{38}$
2^{18}	262,144	2^{35}	$\approx 3.44 \times 10^{10}$	2^{128}	$\approx 3.40 \times 10^{38}$

- ***ASCII***

`'0'`=48, `'A'`=65, `'a'`=97 (`'a'-'A'`=32, es decir, el bit 5).

- ***Constantes reales***

$\pi \approx 3.14159265358979$, $e \approx 2.71828182845905$, $\sqrt{2} \approx 1.41421356237$
$\varphi = \frac{1+\sqrt{5}}{2} \approx 1.6180339887$, $\ln 2 \approx 0.693147$, $\gamma \approx 0.5772156649$.
`acos(-1)` da π con precisión de `double`.

- ***Complejidad según n (límite $\approx 10^8$ ops simples/s)***

$n \le 10$	$O(n!)$	$n \le 5{,}000$	$O(n^2)$
$n \le 20$–25	$O(2^n \cdot n)$	$n \le 10^5$–2×10^5	$O(n \log n)$
$n \le 100$	$O(n^4)$	$n \le 10^6$–10^7	$O(n)$
$n \le 500$	$O(n^3)$	$n \le 10^{18}$	$O(\log n)$ o $O(1)$

- ***Factoriales y Fibonacci (máximo que cabe)***

$12! = 479{,}001{,}600$ es el mayor en `int`; $13! >$ `INT_MAX`.
$20! = 2{,}432{,}902{,}008{,}176{,}640{,}000$ es el mayor en `long long`.
$F_{46} = 1{,}836{,}311{,}903$ mayor en `int`;
$F_{92} = 7{,}540{,}113{,}804{,}746{,}346{,}429$ mayor en `long long`.
$10! = 3{,}628{,}800$,
Catalan: $C_{15} = 9{,}694{,}845$, $C_{19} = 1{,}767{,}263{,}190$ (mayor en `int`).

- ***Máximo número de divisores***

$n \le 10^6$: 240 (720720) $n \le 10^9$: 1344 (735134400) $n \le 10^{18}$: 103,680 (897612484786617600).

- ***Primos y módulos comunes***

$10^9 + 7$, $998{,}244{,}353 = 119 \cdot 2^{23} + 1$ (NTT), $10^9 + 9$.

- ***Cantidades de primos $\le n$:***

n	$\pi(n)$	n	$\pi(n)$
10^2	25	10^9	50,847,534
10^3	168	10^{10}	455,052,511
10^4	1,229	10^{11}	4,118,054,813
10^5	9,592	10^{12}	37,607,912,018
10^6	78,498	10^{13}	346,065,536,839
10^7	664,579	10^{14}	3,204,941,750,802
10^8	5,761,455	10^{15}	29,844,570,422,669

▶ Equivalencias

En general, para enteros **positivos** `int a,b,x` (o `long long`):

`if( a*b >x ) ≡ if( a >x / b )`

Con `>=` `if( a*b >= x ) ≡`

`if(a >= (x + b - 1) / b) ≡ if(a >(x - 1) / b)`

- ***Aritmética entera***
- Techo (positivos): con $a \ge 1$

$\lceil a/b \rceil$ = `(a + b - 1) / b` = `(a - 1) / b + 1`

- Módulo no negativo: `((a% m) + m)% m`. En C++ `-7 / 2 = -3` (trunca hacia 0) y `-7% 2 = -1`.
- Resta modular: `(a - b + m)% m`.
- Inverso modular (m primo): $a^{-1} \equiv a^{m-2}$ (mód m).
- `lcm(a,b) = a / gcd(a,b) * b` (div evitar overflow).
- Overflow de suma: `a + b >MAX` ≡ `a >MAX - b`.
- Overflow de producto: equiv previa o `__int128`.
- `(a + b) / 2` puede desbordar; usar `a + (b - a) / 2`.

▶ Memoria

Unidades. 1 KB $= 2^{10}$ B, 1 MB $= 2^{20}$ = B, 1 GB $= 2^{30}$ B.
Aprox: 1 MB $\approx 10^6$ B 2^{18} ints = 1 MB, 2^{20} ints = 4 MB.
Tamaño de tipos (x86-64, GCC).

`char`, `bool`	1 B	`long long`, `double`	8 B
`short`	2 B	`long double`, `__int128`	16 B
`int`, `float`	4 B	puntero (`int*`)	8 B
`pair<int,int>`	8 B	`pair<ll,ll>`	16 B
`array<int,3>`	12 B	`array<ll,3>`	24 B

Aprox: En la práctica con 256 MB: $\approx 5 \times 10^7$ ints o $\approx 2.5 \times 10^7$ long long.
De n elementos a MB.

n	1 B (`char`)	4 B (`int`)	8 B (`ll`)	16 B
10^5	0.1 MB	0.4 MB	0.8 MB	1.5 MB
10^6	0.95 MB	3.8 MB	7.6 MB	15.3 MB
10^7	9.5 MB	38.1 MB	76.3 MB	152.6 MB
10^8	95.4 MB	381.5 MB	762.9 MB	1,526 MB

De MB a n elementos.

Límite	`char` (1 B)	`int` (4 B)	`ll` (8 B)	16 B
64 MB	6.7×10^7	1.68×10^7	8.4×10^6	4.2×10^6
128 MB	1.34×10^8	3.36×10^7	1.68×10^7	8.4×10^6
256 MB	2.68×10^8	6.71×10^7	3.36×10^7	1.68×10^7
512 MB	5.37×10^8	1.34×10^8	6.71×10^7	3.36×10^7
1024 MB	1.07×10^9	2.68×10^8	1.34×10^8	6.71×10^7

IDEAS <- <- <-

▶ DP

Elegir el estado No siempre pienses en prefijos $[0, i]$; considera también sufijos.
- $dp[i][\dots]$ = es posible sobrevivir las primeras i horas, $[1, i]$.
- $dp[i][\dots]$ = es posible sobrevivir desde el fin de la hora i hasta el final, $(i, n+1]$.

Transiciones sobre el último elemento Revisar si ...
- se usa el último elemento o no.
- se usa el último elemento un número par o impar de veces.

Distinguir entre dos formulaciones $dp[\dots][i]$:
- $dp[\dots][i]$ = ya completé los primeros i elementos.
- $dp[\dots][i]$ = el i-ésimo elemento queda completado/terminado.

Probar quien es el ultimo elemento Si $n \leq 20$ y el orden importa, la transición desde $dp[mask]$ decide qué elemento se agrega *al final*.

▶ Búsqueda binaria

Búsqueda binaria sobre la respuesta Lograr S en a lo sumo K pasos: binaria sobre S. Para S fijo, calcular $f(S)$, la cantidad mínima de pasos para obtener S, y buscar máx$\{S : f(S) \leq K\}$. **Requisito:** f debe ser monótona (si se puede lograr S, se puede lograr cualquier $S' < S$).

▶ Ideas más generales

Ciclos Avanzar un paso a la vez, no varios.

Local Pasar de condiciones locales a condiciones globales.

Independencia (no probabilística): decisiones o eventos que no se afectan entre sí, por ejemplo poner G y T.

Bases o generadores:
- una base B de un espacio vectorial;
- los generadores de una σ-álgebra, $\sigma(\mathcal{C})$.

Informacion En el estado actual, ¿qué información se puede olvidar y cuál es necesario conservar para resolver el problema?

▶ Ideas especificas

No buscar todo Los problemas "**South America** 2025 - Brazil First Phase **K** - Knockout" y "(**NWERC** 2025) **D** Dreamcatcher" mostraron que no hace falta probar toda la fuerza bruta si no algunos "lugares cercanos a donde parece que debería estar el maximo". En lugar de probar $O(N)$ probar una **cierta cantidad**, o empezar a probar **desde "el lugar donde este el maximo"**

OBSERVACIONES AL IMPLEMENTAR

Productos en `int` Castea *antes* de multiplicar: `(ll)(i-1)*i/2`. Castear el resultado ya no sirve.

Saturación Si un valor se acota a `INF`, trátalo como "$\geq$ INF", nunca como 0. Guarda el valor saturado *antes* de cualquier `break`.

Cortes en DP Cortar el cálculo solo es válido si lo que no se calcula nunca se lee después. Verifica que los índices leídos siempre queden dentro del rango calculado.

Restas con saturación Una resta sobre valores saturados es inexacta. Asegura que el sustraendo siempre sea exacto, o usa `__int128` para los intermedios.

Intermedios anchos Calcula en `__int128` y satura al guardar. La suma de dos valores $\leq 4 \cdot 10^{18}$ cabe en `ll`, pero una suma con resta intermedia puede no caber.

Monotonía como test Si sabes que una cantidad es monótona, ponle un `assert`. Un salto raro suele indicar un bug de truncamiento y no de overflow.

Depuración Imprime en puntos de transición (donde cambia el comportamiento) y no en intervalos regulares.

Memoria Si la recurrencia solo usa la fila anterior, usa dos filas. Una tabla de `ll` de $2 \cdot 10^5 \times 196$ pesa unos 313 MB.

COSAS TRIVIALES A TOMAR EN CUENTA

▶ ¿Qué hacer si da WA (Wrong Answer)?
- Revisar que se esta imprimiendo. Otros couts, formato.
- Quitar `ifstream cin("in.txt");` y revisar `DEBUG`
- **Revisar Overflow:** revisar los tipos de datos (`int`, `int64_t`).
- **Revisar línea a línea:** asegúrate de que cada linea si este correcta, por ejemplo `fact[n]` y no solo `n`.
- **Casos extremos:** prueba con casos límites, como $N = 1$, o valores muy grandes.
- Los fors estan bien: no se sale o empieza antes. Por ejemplo en divisibildiad y primos tener cuidado si se inicia en `i=1` en lugar de `i=2`
- Revisar los limites del problema y los limites que se pusieron en el codigo tipo `MAXN=` , `r=` , etc.
- Revisar como llenas la dp, o como llenas arreglos, sobretodo si es inplace, no es lo mismo $0 \to n$ que $n \to 0$.
- ...

▶ ¿Qué hacer si da RT (Runtime error)?
- Declarar arreglos fuera del main.
- Si usas modulos, ojo con las restas aunque creas que da postivio. $A > B$ **NO** implica $(A \mod m) > (B \mod m)$.
- ...

▶ ¿Qué hacer si da TLE (Time Limit Excede)?
- Quitar el Print Debugger
- Revisar que si este usando

```
ios_base::sync_with_stdio(0);
    cin.tie(0);
    cout.tie(0);
```

- Ver que nada se esta ciclando
-

▶ ¿Qué hacer si da MLE (Memory Limit Exceeded) ?

Para medir el consumo pico de memoria RAM

```
g++ -O3 j.cpp -o j
/usr/bin/time -v ./j < jenga.in
```

En la salida generada, busca la línea:

`Maximum resident set size (kbytes): [valor]`

Divide el valor obtenido entre 1024 para convertirlo a megabytes (MB) y compararlo con el límite del problema.

▶ Memoria

`T arr[N]` $N \times \text{sz(T)}$ – La más eficiente (contigua).

`vector<T>` $N \times \text{sz(T)}$ – +32 bytes de overhead del objeto.

`vector<bool>` $\approx N/8$ bytes – Optimizado (1 bit por elemento).

`bitset<N>` $\approx N/8$ bytes – Estático, el más ligero de todos.

`set<T>` $\approx 48N$ bytes – Árbol: 3 punteros + color + dato + padding.

`map<K, V>` $\approx 48N$ bytes – Similar al set, guarda pares `std::pair`.

`unordered_set<T>` $\approx 32N$ bytes (Tabla Hash: ptr por nodo + overhead de buckets)

`list<T>` $\approx 24N + \text{sz(T)} \times N$ – List doblement enlazada (2 punteros).

`deque<T>` $\approx N \times \text{sz(T)}$(Fragment en bloques,más pesada que vector)

`priority_queue<T>` $N \times \text{sz(T)}$ – Usa un `vector` por debajo (Heap).

`set<T>` 32 a 48 bytes + sz(T) (Memoria por nodo, aprox. 64-bit).

`map<K, V>` 32 a 48 bytes + sz(K) + sz(V)

`unordered_set<T>` 24 a 32 bytes + sz(T)

SOBRECARGAR

```
struct T {
    ll a, b;
    // ctor por defecto + parametros
    T(ll a_ = 0, ll b_ = 0) : a(a_), b(b_) {}
    //  compuestos: miembro, modifican *this y devuelven T&
    T& operator+=(const T& o)
        { a += o.a; b += o.b; return *this; }
    T& operator-=(const T& o)
        { a -= o.a; b -= o.b; return *this; }
    T& operator*=(ll k)  { a *= k;   b *= k;   return *this; }
    // --- binarios: friend, copia por valor + compuesto ---
    friend T operator+(T x, const T& y) { return x += y; }
    friend T operator-(T x, const T& y) { return x -= y; }
    friend T operator*(T x, ll k)       { return x *= k; }
    // k*x conmutado
    friend T operator*(ll k, T x)       { return x *= k; }
    // unario
    T operator-() const { return T(-a, -b); }
    // ++t  (prefijo, devuelve referencia)
    T& operator++()    { ++a; ++b; return *this; }
    // t++ (postfijo, devuelve la copia vieja)
    T  operator++(int) { T r = *this; ++*this; return r; }
    //  comparacion (set, map, sort, priority_queue usan < )
    bool operator<(const T& o) const
        { return tie(a, b) < tie(o.a, o.b); }
    bool operator==(const T& o) const
        { return a == o.a && b == o.b; }
    bool operator!=(const T& o) const
        { return !(*this == o); }
    // C++20: reemplaza los 3 anteriores (y >, <=, >=) con:
    auto operator<=>(const T& o) const
        { return tie(b, a) <=> tie(o.b, o.a); }
    bool operator==(const T& o) const = default;
    // --- otros utiles ---
    // t(x) como funcion
    ll operator()(ll x) const { return a * x + b; }
    // t[0], t[1]
    ll& operator[](int i) { return i ? b : a; }
    // if(t)
    explicit operator bool() const { return a || b; }
    // --- I/O: free functions, NO miembros ---
    friend ostream& operator<<(ostream& os, const T& t)
        { return os << t.a << ' ' << t.b; }
    friend istream& operator>>(istream& is, T& t)
        { return is >> t.a >> t.b; }
};
// hash para unordered_map/set
// (fuera del struct, requiere operator==)
template<> struct std::hash<T> {
    size_t operator()(const T& t) const {
        return hash<ll>()(t.a * 1000003 ^ t.b);
    }
};
```

TEORÍA DE CONJUNTOS

Universo U, complemento $\overline{A} = U \setminus A$, diferencia simétrica $A \triangle B = (A \setminus B) \cup (B \setminus A)$ (es el $\oplus$ de la hoja de lógica). Toda identidad se prueba *elemento por elemento* o con indicadores (§2).

► Diccionario conjuntos – lógica – bitwise

Conjuntos	Lógica ($x \in \cdot$)	Máscara
$A \cup B$	$p \vee q$	`a|b`
$A \cap B$	$p \wedge q$	`a&b`
$\overline{A}$	$\neg p$	`~a & full`
$A \setminus B$	$p \wedge \neg q$	`a & ~b`
$A \triangle B$	$p \oplus q$	`a^b`
$A \subseteq B$	$p \Rightarrow q$	`(a&b)==a`
$\|A\|$	—	`popcount(a)`

► Álgebra de Boole de $\mathcal{P}(U)$

- $A \cup A = A \cap A = A$ (idempotencia); conmutatividad y asociatividad de $\cup, \cap, \triangle$.
- $A \cup \varnothing = A$, $A \cap U = A$, $A \cup U = U$, $A \cap \varnothing = \varnothing$.
- Absorción: $A \cup (A \cap B) = A$, $A \cap (A \cup B) = A$.
- Complemento: $A \cup \overline{A} = U$, $A \cap \overline{A} = \varnothing$, $\overline{\overline{A}} = A$.
- $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$.
- $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$.
- De Morgan: $\overline{A \cup B} = \overline{A} \cap \overline{B}$, $\overline{A \cap B} = \overline{A} \cup \overline{B}$.
- $A \subseteq B \iff A \cup B = B \iff A \cap B = A \iff A \setminus B = \varnothing \iff \overline{B} \subseteq \overline{A}$.

Diferencia:

- $A \setminus B = A \cap \overline{B} = A \setminus (A \cap B)$.
- $(A \setminus B) \setminus C = A \setminus (B \cup C)$.
- $A \setminus (B \setminus C) = (A \setminus B) \cup (A \cap C)$.
- $A \setminus (B \cup C) = (A \setminus B) \cap (A \setminus C)$.
- $A \setminus (B \cap C) = (A \setminus B) \cup (A \setminus C)$.
- $(A \cup B) \setminus C = (A \setminus C) \cup (B \setminus C)$.
- $A \cap (B \setminus C) = (A \cap B) \setminus C$.

► Diferencia simétrica

- $A \triangle B = (A \cup B) \setminus (A \cap B)$.
- $A \triangle \varnothing = A$, $A \triangle A = \varnothing$; por tanto $A \triangle B = C \iff A = B \triangle C$.
- $A \cup B = (A \triangle B) \sqcup (A \cap B)$
- $A \triangle (A \cap B) = A \setminus B = (A \cup B) \triangle B$
- $(A \cap B) \triangle (A \cup B) = A \triangle B$
- $\overline{A} \triangle \overline{B} = A \triangle B$, $\overline{A \triangle B} = \overline{A} \triangle B$.
- $\cap$ distribuye sobre $\triangle$: $A \cap (B \triangle C) = (A \cap B) \triangle (A \cap C)$
- $(\mathcal{P}(U), \triangle, \cap)$ es un anillo booleano.
- $\cup$ **no** distribuye sobre $\triangle$. Contraejemplo: $A = \{1\}$, $B = C = \{2\}$: $A \cup (B \triangle C) = \{1\}$ pero $(A \cup B) \triangle (A \cup C) = \varnothing$.
- $|A \triangle C| \le |A \triangle B| + |B \triangle C|$
- distancia de Hamming: $|A \triangle B| = \text{popcount}(a \oplus b)$)
- $\triangle_{i=1}^{n} A_i = \{x : \text{ } x \text{ está en un número impar de los } A_i\}$.

► Indicadores y cardinalidades

Con $\mathbf{1}_A(x) \in \{0,1\}$ y $|A| = \sum_x \mathbf{1}_A(x)$:

- $\mathbf{1}_{A \cap B} = \mathbf{1}_A \, \mathbf{1}_B$, $\mathbf{1}_{\overline{A}} = 1 - \mathbf{1}_A$.
- $\mathbf{1}_{A \cup B} = \mathbf{1}_A + \mathbf{1}_B - \mathbf{1}_A \, \mathbf{1}_B$.
- $\mathbf{1}_{A \triangle B} = \mathbf{1}_A + \mathbf{1}_B - 2\,\mathbf{1}_A \, \mathbf{1}_B \equiv \mathbf{1}_A + \mathbf{1}_B$ (mód 2).
- $|A| + |B| = |A \cup B| + |A \cap B|$ (análogo $a + b = a|b + a\&b$).
- $|A \triangle B| = |A| + |B| - 2\,|A \cap B|$ (análogo $a + b = a \oplus b + 2(a\&b)$).
- $|A \setminus B| = |A| - |A \cap B|$.
- $\max(|A|, |B|) \le |A \cup B| \le |A| + |B|$.
- $|A \cap B| \ge |A| + |B| - |U|$.

► Uniones e intersecciones generalizadas

- $\bigcup_{i \in I} A_i = \{x : \exists i \in I, \ x \in A_i\}$,
- $\bigcap_{i \in I} A_i = \{x \in U : \forall i \in I, \ x \in A_i\}$.
- Convención: $\bigcup_{\varnothing} = \varnothing$, $\bigcap_{\varnothing} = U$
- el $\exists$ sobre vacío es falso, el $\forall$ es verdadero.
- De Morgan: $\overline{\bigcup_i A_i} = \bigcap_i \overline{A_i}$, $\overline{\bigcap_i A_i} = \bigcup_i \overline{A_i}$ $(\neg\exists \equiv \forall\neg)$.
- $B \cap \bigcup_i A_i = \bigcup_i (B \cap A_i)$, $B \cup \bigcap_i A_i = \bigcap_i (B \cup A_i)$.
- $B \setminus \bigcup_i A_i = \bigcap_i (B \setminus A_i)$, $B \setminus \bigcap_i A_i = \bigcup_i (B \setminus A_i)$.
- **Intersección de uniones:**

$$\Big(\bigcup_{i \in I} A_i\Big) \cap \Big(\bigcup_{j \in J} B_j\Big) = \bigcup_{(i,j) \in I \times J} (A_i \cap B_j)$$

- **Unión de intersecciones:**

$$\Big(\bigcap_{i \in I} A_i\Big) \cup \Big(\bigcap_{j \in J} B_j\Big) = \bigcap_{(i,j) \in I \times J} (A_i \cup B_j)$$

- **No conmutan:** $\bigcup_i \bigcap_j A_{ij} \subseteq \bigcap_j \bigcup_i A_{ij}$, y la inclusión inversa es falsa.
- **Disjuntificación:** con $B_i = A_i \setminus \bigcup_{j<i} A_j$ los B_i son disjuntos dos a dos, $B_i \subseteq A_i$ y $\bigcup_i A_i = \bigsqcup_i B_i$; luego $\left|\bigcup A_i\right| = \sum |B_i|$.
- **Partición:** si $\{P_k\}$ particiona U, $X = \bigsqcup_k (X \cap P_k)$ y $|X| = \sum_k |X \cap P_k|$.
- **Subaditividad:** $\left|\bigcup_i A_i\right| \le \sum_i |A_i|$. Igualdad ssi disjuntos dos a dos.
- $\limsup A_n = \bigcap_n \bigcup_{k \ge n} A_k = \{x : x \in A_k \text{ para infinitos } k\}$.
- $\liminf A_n = \bigcup_n \bigcap_{k \ge n} A_k = \{x : x \in A_k \ \forall k \text{ salvo finitos}\}$
- $\liminf A_n \subseteq \limsup A_n$.

► Inclusión–exclusión

- $\left|\bigcup_{i=1}^{n} A_i\right| = \sum_{\varnothing \ne S \subseteq [n]} (-1)^{|S|+1} \left|\bigcap_{i \in S} A_i\right|$.
- $\left|U \setminus \bigcup_i A_i\right| = \sum_{S \subseteq [n]} (-1)^{|S|} \left|\bigcap_{i \in S} A_i\right|$, con $\bigcap_{\varnothing} = U$.
- Sobreyecciones $[n] \to [m]$: $\sum_{k=0}^{m} (-1)^k \binom{m}{k} (m-k)^n$.
- Desarreglos: $D_n = \sum_{k=0}^{n} (-1)^k \, n!/k!$.
- Coprimos con m hasta N: $\sum_{d \mid \text{rad}(m)} \mu(d) \lfloor N/d \rfloor$. Donde

$$m = p_1^{e_1} \cdots p_r^{e_r} \implies \text{rad}(m) = p_1 p_2 \cdots p_r.$$

► Imágenes y preimágenes ($f : X \to Y$)

- $f^{-1}\big(\bigcup B_i\big) = \bigcup f^{-1}(B_i)$, $f^{-1}\big(\bigcap B_i\big) = \bigcap f^{-1}(B_i)$.
- $f^{-1}(\overline{B}) = \overline{f^{-1}(B)}$, $f^{-1}(B \setminus C) = f^{-1}(B) \setminus f^{-1}(C)$.
- $f\big(\bigcup A_i\big) = \bigcup f(A_i)$.
- $f\big(\bigcap A_i\big) \subseteq \bigcap f(A_i)$.
- $f(A \setminus B) \supseteq f(A) \setminus f(B)$, con igualdad si f es inyectiva.
- $f(f^{-1}(B)) = B \cap f(X)$, $f^{-1}(f(A)) \supseteq A$.

► Producto cartesiano y conjunto potencia

- $(A \times B) \cap (C \times D) = (A \cap C) \times (B \cap D)$; $|A \times B| = |A| \, |B|$.
- $A \times (B \cup C) = (A \times B) \cup (A \times C)$, $A \times (B \cap C) = (A \times B) \cap (A \times C)$.
- $(A \times B) \cup (C \times D) \subsetneq (A \cup C) \times (B \cup D)$ en general.
- $\mathcal{P}(A) \cap \mathcal{P}(B) = \mathcal{P}(A \cap B)$.
- $\mathcal{P}(A) \cup \mathcal{P}(B) \subseteq \mathcal{P}(A \cup B)$, estricto salvo que $A \subseteq B$ o $B \subseteq A$.

► Trucos frecuentes en problemas

- Elementos que aparecen una cantidad impar de veces: XOR (o $\triangle$) de todos los A_i.
- Doble conteo: $\sum_i |A_i| = \sum_x \#\{i : x \in A_i\}$.
- Pares intersecantes: $\#\{(i,j) : A_i \cap A_j \ne \varnothing\} = \text{total} - \#\{\text{disjuntos}\}$.
- Unión de intervalos: ordenar y barrer; la longitud de la unión **no** es la suma de longitudes (disjuntificar).
- Antes de intercambiar $\bigcup\bigcap$, revisa que los cuantificadores no conmutan (fuente típica de bugs).
- $\overline{A}$ y $\bigcap_{\varnothing}$ dependen de U; en máscaras, el complemento exige `& full`.

SUBCONJUNTOS Y FUNCIONES SOBRE MÁSCARAS

- $|\mathcal{P}(A)| = 2^{|A|}$; hay $\binom{n}{k}$ subconjuntos de tamaño k; $\sum_k \binom{n}{k} = 2^n$; $\sum_k (-1)^k \binom{n}{k} = 0$ para $n \ge 1$.
- Pares $A \subseteq B \subseteq [n]$: 3^n (cada elemento: fuera de B, en $B \setminus A$, o en A). De aquí el costo $O(3^n)$ de enumerar los submasks de todas las máscaras.
- Cadenas $A_1 \subseteq A_2 \subseteq \cdots \subseteq A_k \subseteq [n]$: $(k+1)^n$.
- **Zeta** (subconjuntos): $\hat{f}(S) = \sum_{T \subseteq S} f(T)$, en $O(n 2^n)$ procesando un bit a la vez.
- Convolución OR: $\widehat{f *_{\cup} g} = \hat{f} \cdot \hat{g}$ (producto puntual tras la zeta).
- Convolución AND: igual, con la transformada sobre supersets. Convolución XOR: Walsh–Hadamard.

► Mobius General

Para cualquier conjunto con orden parcial $\le$ existe una funcion $\mu(x, y)$ que depende solo del orden

Suma hacia abajo	Suma hacia arriba
Divisibilidad	
$g(n) = \sum_{d \mid n}^{\lfloor V/d \rfloor} f(d)$	$g(d) = \sum_{k \ge 1}^{\lfloor V/d \rfloor} f(kd)$
$f(n) = \sum_{d \mid n} \mu(d)\, g(n/d)$	$f(d) = \sum_{k \ge 1} \mu(k)\, g(kd)$
Subconjuntos	
$\hat{f}(S) = \sum_{T \subseteq S} f(T)$	$\check{f}(S) = \sum_{T \supseteq S} f(T)$
$f(S) = \sum_{T \subseteq S} (-1)^{\lvert S \setminus T \rvert} \hat{f}(T)$	$f(S) = \sum_{T \supseteq S} (-1)^{\lvert T \setminus S \rvert} \check{f}(T)$
General (POSET)	
$g(x) = \sum_{y \le x} f(y)$	$g(x) = \sum_{y \ge x} f(y)$
$f(x) = \sum_{y \le x} \mu(y, x)\, g(y)$	$f(x) = \sum_{y \ge x} \mu(x, y)\, g(y)$

PROBABILIDAD Y PROCESOS ESTOCÁSTICOS

▶ Distribuciones Discretas Fundamentales

• ***Distribución de Bernoulli***

Modela un único ensayo aleatorio con dos posibles resultados: Éxito (1) con probabilidad p, o Fracaso (0) con probabilidad $1-p$.

- **Función de Masa de Probabilidad (PMF):** $P(X = k) = p^k(1-p)^{1-k}, \quad k \in \{0,1\}$
- **Esperanza:** $E[X] = 1 \cdot p + 0 \cdot (1-p) = p$
- **Varianza:** $\text{Var}(X) = p(1-p)$
- **Variable Indicadora:** Si X_i es la variable indicadora de que ocurra un evento A, entonces $E[X_i] = P(A)$.

• ***Distribución Binomial***

Modela la cantidad de éxitos k obtenidos al realizar n ensayos independientes de Bernoulli de probabilidad p.

- **PMF:** $P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}, \quad k \in \{0, 1, \ldots, n\}$
- **Esperanza:** Por linealidad de la esperanza ($X = \sum_{j=1}^{n} X_j$):

$$E[X] = \sum_{j=1}^{n} E[X_j] = n \cdot p$$

• ***Distribución Geométrica y Tiempo de Parada***

Modela el tiempo de parada T, definido como el número de ensayos de Bernoulli independientes con probabilidad de éxito p necesarios hasta obtener el **primer éxito**.

- **PMF:** $P(T = i) = (1-p)^{i-1}p, \quad i \in \{1, 2, 3, \ldots\}$
- **Esperanza (Tiempo Esperado de Parada):**

$$E[T] = \sum_{i=1}^{\infty} i \cdot P(T = i) = \sum_{i=1}^{\infty} i(1-p)^{i-1}p = \frac{1}{p}$$

- **Interpretación de Casos Favorables entre Casos Totales:** Si el espacio muestral en i pasos tiene M^i secuencias equiprobables y en los primeros $i-1$ pasos hay F opciones de fracaso y en el i-ésimo hay S opciones de éxito (donde $p = S/M$ y $1-p = F/M$):

$$P(T = i) = \frac{F^{i-1} \cdot S}{M^i} = \left(\frac{F}{M}\right)^{i-1} \left(\frac{S}{M}\right)$$

▶ Técnicas de Análisis en Procesos Estocásticos y Cadenas de Markov

• ***Análisis del Primer Paso (First-Step Analysis)***

Para encontrar el tiempo esperado de llegada $E[u]$ desde un estado u hacia un estado terminal, se condiciona sobre la primera transición saliente:

$$E[u] = 1 + \sum_{v} P(u \to v) \cdot E[v]$$

Donde el $+1$ representa el costo/tiempo unitario consumido al realizar el paso actual.

• ***Despeje de Auto-ciclos (Self-Loops)***

Si un estado u transiciona a sí mismo con probabilidad $p_{self} = P(u \to u)$:

$$E[u] = 1 + p_{self} \cdot E[u] + \sum_{v \neq u} P(u \to v)E[v]$$

Despejando $E[u]$ algebraicamente para evitar recursión infinita:

$$E[u] = \frac{1 + \sum_{v \neq u} P(u \to v)E[v]}{1 - p_{self}}$$

• ***Linealidad de la Esperanza y Descomposición***

Para cualquier conjunto de variables aleatorias $X_1, X_2, \ldots, X_n$ (sean independientes o dependientes):

$$E\left[\sum_{j=1}^{n} X_j\right] = \sum_{j=1}^{n} E[X_j]$$

Permite calcular el tiempo esperado total de transformar N elementos como N veces el tiempo esperado de transformar un solo elemento, siempre que el proceso afecte a cada elemento de manera independiente.

▶ Ideas:

Maximo $max_i \leq c \iff \forall i \; X_i \leq c$

El proceso primero elige una situación o estado y luego evoluciona de manera distinta según esa elección.

$$\mathbb{E}[X] = \sum_{s} \Pr(S = s)\mathbb{E}[X \mid S = s]$$

PLANTILLA PYTHON

```python
# librerias
import os, random, re, sys
from collections import deque
# limite memoria
sys.setrecursionlimit(200000)
# --- --- --- INTPUT & FAST --- --- ---
sys.stdin = open("in.txt", 'r', encoding='utf-8') # comentar
input = sys.stdin.readline
# --- --- --- DEBUG --- --- ---
DEB = False # True False
def debug(*args, **kwargs):
    if DEB:
        print("[DEBUG]", *args, **kwargs, file=sys.stderr)
# --- --- --- Leer y resolver --- --- ---
def solve():
    # separar multiples enteros en una línea
    n,m  = map(int, input().split())
    # leer y quitar salto '\n'
    n = int( input().rstrip() )
    # leer una lista de enteros
    h = list( map(int, input().rstrip().split()) )
    # cadena de texto sin espacios
    s = input().rstrip()
    # escribir codigo aqui
# --- --- --- MAIN --- --- ---
if __name__ == "__main__":
    t = int( input() )
    for i in range(t):
        solve()
```

▶ Fenwick Tree

Query puntual, actualizaciones en rangos. $O(\log n)$

```python
class Fenwick:
    def __init__( self, n ):
        self.bit = [0] * (n+1)
        self.n = n
    def add( self, i, val ): # agrega puntual
        i += 1
        while i <= self.n:
            self.bit[i] += val
            i += i & (-i)
    def add_rango( self, l, r, val): # en rango
        self.add( l, val)
        if r+ 1 < self.n :
            self.add( r+1, -val )
    def query_puntual( self, i): # query para un solo punto
        res = 0
        i += 1
        while i > 0:
            res += self.bit[i]
            i -= i & (-i)
        return res
def solve():
    n,m  =   map(int, input().split())
    arbol = Fenwick(n)
    for _ in range(m):
        a,b, val =   map(int, input().split())
        arbol.add_rango( l=a-1, r=b-1, val = val)
    for i in range(n):
        print( arbol.query_puntual(i) )
```

MODELADO

▸ Distribuciones

Discretas

Bernoulli $X \sim \mathrm{Ber}(p)$, $p \in (0,1)$; $\mathcal{S}_X = \{0,1\}$; $f(x) = p^x(1-p)^{1-x}$; $\mathbb{E} = p$; $\mathrm{Var} = p(1-p)$; $M(t) = 1-p+pe^t$.

Binomial $X \sim \mathrm{Bin}(n,p)$; $\mathcal{S}_X = \{0,\dots,n\}$; $f(x) = \binom{n}{x} p^x(1-p)^{n-x}$; $\mathbb{E} = np$; $\mathrm{Var} = np(1-p)$; $M(t) = (1-p+pe^t)^n$.

Geométrica (ensayos hasta el 1.er éxito) $X \sim \mathrm{Geo}(p)$, $p \in (0,1]$; $\mathcal{S}_X = \{1,2,\dots\}$; $f(x) = (1-p)^{x-1}p$; $\mathbb{E} = \frac{1}{p}$; $\mathrm{Var} = \frac{1-p}{p^2}$; $M(t) = \frac{pe^t}{1-(1-p)e^t}$, $t < -\ln(1-p)$.

Bin. Negativa (ensayos hasta el r-ésimo éxito) $X \sim \mathrm{NB}(r,p)$; $\mathcal{S}_X = \{r, r+1, \dots\}$; $f(x) = \binom{x-1}{r-1} p^r(1-p)^{x-r}$; $\mathbb{E} = \frac{r}{p}$; $\mathrm{Var} = \frac{r(1-p)}{p^2}$; $M(t) = \left(\frac{pe^t}{1-(1-p)e^t}\right)^r$, $t < -\ln(1-p)$.

Poisson $X \sim \mathrm{Poi}(\lambda)$, $\lambda > 0$; $\mathcal{S}_X = \{0,1,\dots\}$; $f(x) = \frac{e^{-\lambda}\lambda^x}{x!}$; $\mathbb{E} = \mathrm{Var} = \lambda$; $M(t) = e^{\lambda(e^t-1)}$.

Hipergeom. $X \sim \mathrm{Hyp}(N,K,n)$; $\mathcal{S}_X = \{\max(0, n-N+K), \dots, \min(n,K)\}$; $f(x) = \frac{\binom{K}{x}\binom{N-K}{n-x}}{\binom{N}{n}}$; $\mathbb{E} = \frac{nK}{N}$; $\mathrm{Var} = n\frac{K}{N}(1-\frac{K}{N})\frac{N-n}{N-1}$; $M(t)$ sin forma elemental (${}_2F_1$).

Continuas

Uniforme $X \sim \mathcal{U}(a,b)$; $\mathcal{S}_X = [a,b]$; $f(x) = \frac{1}{b-a}$; $\mathbb{E} = \frac{a+b}{2}$; $\mathrm{Var} = \frac{(b-a)^2}{12}$; $M(t) = \frac{e^{tb}-e^{ta}}{t(b-a)}$ $(t \neq 0)$, $M(0) = 1$.

Exponencial $X \sim \mathrm{Exp}(\lambda)$, $\lambda > 0$; $\mathcal{S}_X = [0,\infty)$; $f(x) = \lambda e^{-\lambda x}$; $\mathbb{E} = \frac{1}{\lambda}$; $\mathrm{Var} = \frac{1}{\lambda^2}$; $M(t) = \frac{\lambda}{\lambda - t}$, $t < \lambda$.

Normal $X \sim \mathcal{N}(\mu, \sigma^2)$, $\sigma > 0$; $\mathcal{S}_X = \mathbb{R}$; $f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-(x-\mu)^2/2\sigma^2}$; $\mathbb{E} = \mu$; $\mathrm{Var} = \sigma^2$; $M(t) = e^{\mu t + \sigma^2 t^2/2}$.

Gamma (forma–tasa) $X \sim \mathrm{Gamma}(\alpha,\beta)$; $\mathcal{S}_X = (0,\infty)$; $f(x) = \frac{\beta^\alpha}{\Gamma(\alpha)} x^{\alpha-1}e^{-\beta x}$; $\mathbb{E} = \frac{\alpha}{\beta}$; $\mathrm{Var} = \frac{\alpha}{\beta^2}$; $M(t) = (\frac{\beta}{\beta - t})^\alpha$, $t < \beta$.

Beta $X \sim \mathrm{Beta}(\alpha,\beta)$; $\mathcal{S}_X = [0,1]$; $f(x) = \frac{x^{\alpha-1}(1-x)^{\beta-1}}{B(\alpha,\beta)}$, $B = \frac{\Gamma(\alpha)\Gamma(\beta)}{\Gamma(\alpha+\beta)}$; $\mathbb{E} = \frac{\alpha}{\alpha+\beta}$; $\mathrm{Var} = \frac{\alpha\beta}{(\alpha+\beta)^2(\alpha+\beta+1)}$; $M(t) = {}_1F_1(\alpha; \alpha+\beta; t)$.

▸ Casos de uso y modelación

Guía rápida: señal → modelo "n intentos iguales e independientes, contar éxitos" → Bin; "intentos hasta el primer/r-ésimo éxito" → Geo/NB; "eventos raros en un intervalo", sin n explícito → Poi; "sacar sin reemplazo" → Hyp; "punto/instante al azar" → $\mathcal{U}$; "tiempo hasta que ocurre" con tasa constante → Exp; "tiempo hasta el k-ésimo evento" → Gamma; "suma de muchas partes pequeñas" → $\mathcal{N}$ (aproximación); "p desconocida" o "p uniforme" → Beta (Laplace); "solo importa el orden relativo" → permutación uniforme (intercambiabilidad). **Antes de modelar:** (i) ¿hay independencia o solo intercambiabilidad? (ii) ¿la linealidad de $\mathbb{E}$ resuelve sin conocer la distribución? (iii) ¿se pide exactitud o tolerancia 10^{-6}?

Ber/Bin Señal: n intentos independientes con la misma p; indicador de un evento. **Modelo:** $X = \sum_{j \le n} X_j$, $X_j \overset{iid}{\sim} \mathrm{Ber}(p)$; $\mathbb{P}(X \ge 1) = 1 - (1-p)^n$ (complemento). **Relaciones:** $\mathrm{Bin}(n,p) + \mathrm{Bin}(m,p) = \mathrm{Bin}(n+m,p)$ (indep.); $\approx \mathrm{Poi}(np)$ si $n \to \infty$, $np \to \lambda$; $\approx \mathcal{N}(np, np(1-p))$ si $np(1-p) \gtrsim 10$ (con corrección de continuidad $\pm\frac{1}{2}$); si $X \sim \mathrm{Bin}(n,p)$, $Y \sim \mathrm{Bin}(m,p)$ indep., entonces $X \mid (X+Y=s) \sim \mathrm{Hyp}$. **En CP:** $\mathbb{E}[\sum X_j] = \sum \mathbb{E}[X_j]$ vale *aunque haya dependencia* (la varianza sí exige independencia); pmf con recurrencia $f(k+1) = f(k)\frac{n-k}{k+1}\frac{p}{1-p}$ en $O(n)$, o factoriales precalculados (mod M) en $O(1)$ por término; n grande en reales ⇒ logaritmos de factoriales (`lgamma`).

Geo Señal: "cuántos intentos hasta el primer éxito", "se repite hasta que sale", reinicio tras fallo. **Modelo:** tiempo de espera discreto; $\mathbb{P}(X > k) = (1-p)^k$; es la única discreta sin memoria: $\mathbb{P}(X > m+k \mid X > m) = \mathbb{P}(X > k)$. **Relaciones:** $\min_i \mathrm{Geo}(p_i) \sim \mathrm{Geo}\big(1 - \prod_i (1-p_i)\big)$ (indep.); $\mathrm{Geo}(\lambda/n)/n \to \mathrm{Exp}(\lambda)$. **En CP:** en un DP, el autolazo se despeja como $E = c + (1-p)E + \cdots \Rightarrow E = \frac{c+\dots}{p}$ (no hay que iterar); muestreo por inversión $\lceil \ln U / \ln(1-p) \rceil$.

NB Señal: "intentos hasta acumular r éxitos". **Modelo:** $X = \sum_{j \le r} G_j$, $G_j \overset{iid}{\sim} \mathrm{Geo}(p)$; en la pmf el último ensayo es éxito y los $r-1$ anteriores se reparten entre las $x-1$ posiciones. **Relaciones:** dualidad $\mathbb{P}(\mathrm{NB}(r,p) \le x) = \mathbb{P}(\mathrm{Bin}(x,p) \ge r)$; versión por fracasos $= X - r$ (soporte desde 0; cuidar la convención). **En CP:** la dualidad evita un DP sobre "éxitos acumulados" y da la cola en $O(r)$ sumando una binomial.

Poisson Señal: conteo de eventos raros e independientes en un intervalo, dado el promedio λ pero no n; llegadas a una cola. **Modelo:** $N(t) \sim \mathrm{Poi}(\lambda t)$ con conteos independientes en intervalos disjuntos. **Relaciones:** superposición: $\mathrm{Poi}(\lambda_1) + \mathrm{Poi}(\lambda_2) \sim \mathrm{Poi}(\lambda_1 + \lambda_2)$; *thinning*: conservar cada evento con prob. q da $\mathrm{Poi}(\lambda q)$, y conservados y descartados son independientes; $N_1 \mid (N_1 + N_2 = s) \sim \mathrm{Bin}\big(s, \frac{\lambda_1}{\lambda_1+\lambda_2}\big)$; $\mathrm{Bin}(n,p) \approx \mathrm{Poi}(np)$. **En CP:** recurrencia $f(k+1) = f(k)\frac{\lambda}{k+1}$ y truncar la serie hacia $k \approx \lambda + c\sqrt{\lambda}$; *Poissonización*: con n fijo los conteos por categoría son dependientes, pero con $N \sim \mathrm{Poi}(\lambda)$ son independientes ⇒ se calcula fácil y se des-poissoniza.

Hyp Señal: elegir n de N elementos distinguibles sin reemplazo (cartas, lotes, equipos). **Modelo:** cada elemento queda elegido con prob. $\frac{n}{N}$ por simetría, así que $\mathbb{E}[\#\text{rojos elegidos}] = \frac{nK}{N}$ sin importar la dependencia; la n-ésima extracción es roja con prob. $\frac{K}{N}$ (intercambiabilidad), pero *condicionada a g rojas previas* es $\frac{K-g}{N-(n-1)}$. **Relaciones:** si $N \to \infty$ y $K/N \to p$, $\to \mathrm{Bin}(n,p)$; versión multivariada para varias categorías. **En CP:** $\mathbb{P} = \binom{K}{k}\binom{N-K}{n-k} / \binom{N}{n}$ con factoriales modulares en $O(1)$; para probabilidades de configuraciones, contar casos favorables / totales evita DP.

Uniforme Señal: "punto/instante al azar", sin preferencia; "permutación aleatoria". **Modelo:** probabilidad $\propto$ longitud, área o volumen (problemas geométricos se reducen a áreas en el cuadrado unidad); si solo importa el orden de n uniformes iid continuas, todas las $n!$ permutaciones son equiprobables. **Relaciones:** $U \sim \mathcal{U}(0,1) \Rightarrow F^{-1}(U) \sim F$ (transformada inversa); $-\ln U \sim \mathrm{Exp}(1)$; estadísticos de orden → Beta. **En CP:** "probabilidad de que el elemento j sea el mayor/récord" se resuelve por simetría sin integrar; para áreas complicadas, integral en $O(1)$ o partición en regiones.

Exp Señal: "tiempo hasta que ocurre" con tasa constante; varios procesos compitiendo en paralelo; "falta de memoria". **Modelo:** cadena de Markov en tiempo continuo: en el estado s se espera $\mathrm{Exp}(\Lambda_s)$, $\Lambda_s = \sum \lambda_{s \to s'}$, y se salta a s' con prob. $\frac{\lambda_{s\to s'}}{\Lambda_s}$. **Relaciones:** $\min_i T_i \sim \mathrm{Exp}(\sum \lambda_i)$, y $\mathbb{P}(T_k = \min) = \frac{\lambda_k}{\sum \lambda_i}$ (indep.); el máximo de n iid $\mathrm{Exp}(\lambda)$ tiene $\mathbb{E} = \frac{H_n}{\lambda}$; los tiempos entre llegadas de un proceso de Poisson son Exp (dualidad). **En CP:** se convierte en DP discreto sobre la cadena embebida: $E[s] = \frac{1}{\Lambda_s} + \sum_{s'} \frac{\lambda_{s\to s'}}{\Lambda_s} E[s']$, sin integrales ni reales continuos.

Normal Señal: suma de muchas contribuciones pequeñas e independientes (TLC); enunciado que tolera error $\sim 10^{-6}$ con n enorme. **Modelo:** $\sum X_j \approx \mathcal{N}(n\mu, n\sigma^2)$; estandarizar con $Z = \frac{X-\mu}{\sigma}$ y usar Φ. **Relaciones:** cerrada bajo suma independiente: $\mathcal{N}(\mu_1,\sigma_1^2) + \mathcal{N}(\mu_2,\sigma_2^2) = \mathcal{N}(\mu_1+\mu_2, \sigma_1^2+\sigma_2^2)$; aproxima Bin y Poi; Berry–Esseen: error $O(1/\sqrt{n})$, malo en colas. **En CP:** es una *aproximación*; si piden exactitud o respuesta modular no se usa. $\Phi(x) = \frac{1}{2}\texttt{erfc}(-x/\sqrt{2})$ en C++; para estimar tamaños o cotas es suficiente.

Gamma Señal: "tiempo hasta el k-ésimo evento" en un proceso de Poisson. **Modelo:** Erlang: $T = \sum_{j \le k} \mathrm{Exp}(\lambda)$. **Relaciones:** dualidad $\mathbb{P}(T \le t) = \mathbb{P}(\mathrm{Poi}(\lambda t) \ge k)$ (la integral se vuelve una suma finita de k términos); $\mathrm{Gamma}(\alpha,\beta) + \mathrm{Gamma}(\alpha',\beta) = \mathrm{Gamma}(\alpha+\alpha',\beta)$ (indep., misma tasa); si $X \sim \mathrm{Gamma}(\alpha)$, $Y \sim \mathrm{Gamma}(\beta)$ indep., $\frac{X}{X+Y} \sim \mathrm{Beta}(\alpha,\beta)$ e independiente de $X+Y$. **En CP:** cola exacta en $O(k)$; normalizar n exponenciales iid da un punto uniforme en el simplejo (Dirichlet(1,…,1)).

Beta Señal: probabilidad desconocida p ("uniforme" ⇒ Beta(1,1)); posición del k-ésimo punto de n uniformes. **Modelo:** prior conjugado de Ber/Bin con a éxitos y b fracasos virtuales: $p \mid (g \text{ de } i) \sim \mathrm{Beta}(a+g, b+i-g)$ y $\mathbb{P}(X_{i+1}{=}1 \mid g \text{ de } i) = \frac{a+g}{a+b+i}$ (Laplace si $a{=}b{=}1$: $\frac{g+1}{i+2}$). **Relaciones:** $U_{(k)} \sim \mathrm{Beta}(k, n-k+1)$; generalización a k categorías: Dirichlet; urna de Pólya ≡ Beta–Binomial. **En CP:** DP sobre (i,g) con transición $\frac{a+g}{a+b+i}$ (inversos modulares de $a+b+i$) en $O(N^2)$; si solo importa el total de éxitos con Beta(1,1), es uniforme en $\{0,\dots,n\}$ y no hace falta DP.

Mapa de relaciones $\mathrm{Ber} \xrightarrow{\Sigma} \mathrm{Bin} \to \mathrm{Poi}$ (raro) / $\mathcal{N}$ (TLC); $\mathrm{Geo} \xrightarrow{\Sigma} \mathrm{NB}$; $\mathrm{Exp} \xrightarrow{\Sigma} \mathrm{Gamma}$; $\mathrm{Geo} \xrightarrow{n\to\infty} \mathrm{Exp}$; $\mathrm{Poi} \leftrightarrow \mathrm{Exp}$ (conteo ↔ espera); $\mathrm{Bin} \leftrightarrow \mathrm{NB}$, $\mathrm{Gamma} \leftrightarrow \mathrm{Poi}$ (dualidades de colas); $\mathrm{Hyp} \xrightarrow{N\to\infty} \mathrm{Bin}$; Gamma/Gamma → Beta; Beta ↔ Bin (conjugación) y $U_{(k)} \sim \mathrm{Beta}$; $\mathrm{Exp}(1) = -\ln \mathcal{U}$.

MODELACION AVANZADA

Laplace / Beta–Binomial (1) Contexto. Un evento (servidor falla, aparece un monstruo, moneda cae cara) ocurre con probabilidad fija pero desconocida p; tras i intentos con g éxitos se pide la probabilidad del siguiente, o un DP sobre estados (i, g). **(2) Modelo.** $\theta \sim \text{Beta}(a, b)$, $X_j \mid \theta \overset{\text{iid}}{\sim} \text{Ber}(\theta)$, $S_i = \sum_{j \le i} X_j$ (los X_j son intercambiables, no independientes). **(3) Resultado.** $\theta \mid S_i{=}g \sim \text{Beta}(a+g, b+i-g)$; $\mathbb{P}(X_{i+1}{=}1 \mid S_i{=}g) = \frac{a+g}{a+b+i}$; $\mathbb{P}(S_n{=}k) = \binom{n}{k} \frac{B(a+k,\, b+n-k)}{B(a,b)}$; una secuencia concreta con k éxitos tiene probabilidad $\frac{B(a+k,\, b+n-k)}{B(a,b)}$. Con $a{=}b{=}1$: $\frac{g+1}{i+2}$ y S_n uniforme en $\{0, \dots, n\}$. **(4) Idea.** Integral beta $\int_0^1 \theta^x (1-\theta)^y d\theta = \frac{x!\, y!}{(x+y+1)!}$ (por partes) y Bayes: posterior $\propto$ verosimilitud $\times$ prior; Beta es conjugada de Bernoulli. Alternativa sin integrales: argumento de los huecos con $i+2$ uniformes. **(5) CP.** Conviene cuando "p fija desconocida" y se aprende de los resultados.

Dirichlet–Multinomial (1) Contexto. Igual que Laplace pero con k resultados posibles (dado de k caras con sesgo desconocido, k tipos de enemigos). **(2) Modelo.** $\theta \sim \text{Dir}(\alpha_1, \dots, \alpha_k)$, $X_j \mid \theta \overset{\text{iid}}{\sim} \text{Cat}(\theta)$, conteos c_m tras i observaciones, $\alpha_0 = \sum_m \alpha_m$. **(3) Resultado.** $\theta \mid c \sim \text{Dir}(\alpha + c)$; $\mathbb{P}(X_{i+1}{=}m \mid c) = \frac{\alpha_m + c_m}{\alpha_0 + i}$ (Laplace: $\frac{c_m+1}{i+k}$); $\mathbb{P}(c) = \frac{n!}{\prod_m c_m!} \cdot \frac{\Gamma(\alpha_0)}{\Gamma(\alpha_0+n)} \prod_m \frac{\Gamma(\alpha_m+c_m)}{\Gamma(\alpha_m)}$. **(4) Idea.** Integral de Dirichlet $\int_\Delta \prod \theta_m^{x_m-1} d\theta = \frac{\prod \Gamma(x_m)}{\Gamma(\sum x_m)}$ y conjugación; equivale a una urna de Pólya con α_m bolas de cada color. **(5) CP.** $O(k)$ por consulta o $O(1)$ manteniendo i y c; el estado de un DP es el vector de conteos (compresión: contar solo cuántos colores tienen cada conteo). Conviene con varias categorías y probabilidades desconocidas e intercambiables.

Urna de Pólya (1) Contexto. Se saca una bola, se devuelve y se agregan c bolas del mismo color ("el rico se hace más rico"). **(2) Modelo.** Inicio con a rojas y b azules; R_n = rojas obtenidas en n extracciones. **(3) Resultado.** $\mathbb{P}(\text{roja en } n+1 \mid R_n{=}r) = \frac{a+cr}{a+b+cn}$; una secuencia con r rojas y $n-r$ azules tiene probabilidad $\frac{a^{(r,c)}\, b^{(n-r,c)}}{(a+b)^{(n,c)}}$ con $x^{(m,c)} = x(x+c)\cdots(x+(m-1)c)$. Es exactamente Beta–Binomial con $(a/c,\, b/c)$ y la fracción límite es $\text{Beta}(a/c, b/c)$. **(4) Idea.** Inducción sobre la secuencia: el numerador y el denominador son productos crecientes que no dependen del orden (intercambiabilidad). **(5) CP.** $O(1)$ por paso, DP (n, r) en $O(N^2)$. Conviene cuando la probabilidad de la siguiente extracción es proporcional al conteo acumulado.

Estadísticos de orden y espaciamientos (1) Contexto. Se eligen n puntos al azar en $[0, 1]$ (o una vara se parte en $n+1$ trozos) y se pregunta por el mayor, el menor, el k-ésimo o el trozo más largo. **(2) Modelo.** $U_1, \dots, U_n \overset{\text{iid}}{\sim} \mathcal{U}(0, 1)$, $U_{(1)} \le \cdots \le U_{(n)}$, espaciamientos $D_j = U_{(j)} - U_{(j-1)}$ con $U_{(0)}{=}0$, $U_{(n+1)}{=}1$. **(3) Resultado.** $U_{(k)} \sim \text{Beta}(k,\, n-k+1)$; $\mathbb{E}[U_{(k)}] = \frac{k}{n+1}$; $\text{Var} = \frac{k(n-k+1)}{(n+1)^2(n+2)}$; $\mathbb{E}[\text{máx}] = \frac{n}{n+1}$, $\mathbb{E}[\text{mín}] = \frac{1}{n+1}$, $\mathbb{E}[\text{rango}] = \frac{n-1}{n+1}$; $(D_1, \dots, D_{n+1}) \sim \text{Dir}(1, \dots, 1)$ (intercambiables, cada una con esperanza $\frac{1}{n+1}$); $\mathbb{E}[\text{máx}_j\, D_j] = \frac{1}{n+1} \sum_{j=1}^{n+1} \frac{1}{j}$; $\mathbb{P}(U_{n+1}$ es el k-ésimo menor$) = \frac{1}{n+1}$. Para otra F continua: aplicar F^{-1}. **(4) Idea.** $\mathbb{P}(U_{(k)} \in dx) = \frac{n!}{(k-1)!(n-k)!} x^{k-1}(1-x)^{n-k} dx$ (elegir cuáles $k-1$ caen antes de x, una cae en dx, el resto después); espaciamientos intercambiables por simetría, y $\sum D_j = 1$ da la esperanza. **(5) CP.** $O(1)$. Conviene cuando aparecen máximos/mínimos de uniformes, partir intervalos al azar o "probabilidad de que el nuevo punto sea récord".

Suma de colas y máximo discreto (1) Contexto. Esperanza del máximo de n dados de d caras, o de cualquier v.a. entera no negativa con cola conocida. **(2) Modelo.** $X \ge 0$ entera (o continua). **(3) Resultado.** $\mathbb{E}[X] = \sum_{k \ge 1} \mathbb{P}(X \ge k)$ y $\mathbb{E}[X] = \int_0^\infty \mathbb{P}(X > t)\, dt$; para M = máx de n dados iid de d caras: $\mathbb{E}[M] = \sum_{x=1}^{d} \left(1 - (\frac{x-1}{d})^n\right) = d - \sum_{x=1}^{d-1} (\frac{x}{d})^n$. **(4) Idea.** $X = \sum_{k \ge 1} \mathbf{1}[X \ge k]$ y linealidad (o Fubini en el caso continuo); $\mathbb{P}(M \le x) = (x/d)^n$ por independencia. **(5) CP.** $O(d \log n)$ con exponenciación rápida. Conviene siempre que $\mathbb{P}(X \ge k)$ es más fácil que la pmf.

Coleccionista de cupones generalizado (1) Contexto. Hay n tipos de cupón y cada compra da el tipo i con probabilidad p_i; se pide el número esperado de compras hasta tener todos (o m distintos). **(2) Modelo.** T = primer instante en que todos los tipos aparecieron, $\sum p_i = 1$. **(3) Resultado.** $\mathbb{E}[T] = \int_0^\infty \left(1 - \prod_{i=1}^n (1 - e^{-p_i t})\right) dt = \sum_{\emptyset \ne J \subseteq [n]} \frac{(-1)^{|J|+1}}{p(J)}$, con $p(J) = \sum_{i \in J} p_i$. Caso uniforme: $\mathbb{E}[T] = nH_n$ y $\text{Var}(T) = n^2 \sum_{j=1}^n \frac{1}{j^2} - nH_n$. Para m distintos (uniforme): $\mathbb{E} = \sum_{k=0}^{m-1} \frac{n}{n-k}$. **(4) Idea.** Poissonización: las compras llegan como proceso de Poisson de tasa 1, el tipo i aparece como proceso independiente de tasa p_i, así que el tiempo continuo es $\text{máx}_i \text{Exp}(p_i)$ con $\mathbb{P}(\le t) = \prod_i (1 - e^{-p_i t})$; el tiempo medio entre llegadas es 1 y por Wald $\mathbb{E}[T_{\text{cont}}] = \mathbb{E}[T]$. Expandir el producto da inclusión–exclusión con $\int_0^\infty e^{-p(J)t} dt = \frac{1}{p(J)}$. **(5) CP.** Inclusión–exclusión $O(2^n)$ ($n \lesssim 20$); si los p_i son enteros pequeños, DP sobre la suma $p(J)$ con signo en $O(n \cdot S)$; uniforme $O(n)$. Conviene con probabilidades no uniformes y n pequeño, o con DP por estados "cuántos faltan".

Identidad Min–Max (1) Contexto. Esperanza del tiempo en que *todos* los elementos de un conjunto han ocurrido (cupones, procesos paralelos, lámparas que se apagan). **(2) Modelo.** Variables aleatorias X_i (cualquier dependencia), $S = \{1, \dots, n\}$. **(3) Resultado.** $\text{máx}\, S = \sum_{\emptyset \ne T \subseteq S} (-1)^{|T|+1} \text{mín}\, T$, y por linealidad $\mathbb{E}[\text{máx}\, S] = \sum_{T \ne \emptyset} (-1)^{|T|+1} \mathbb{E}[\text{mín}\, T]$. Generalización: $\mathbb{E}[k\text{-ésimo mayor}] = \sum_T (-1)^{|T|-k} \binom{|T|-1}{k-1} \mathbb{E}[\text{mín}\, T]$. **(4) Idea.** Para reales $x_i \ge 0$ y $t \ge 0$: $\mathbf{1}[\text{máx} > t] = 1 - \prod_i (1 - \mathbf{1}[x_i > t]) = \sum_{T \ne \emptyset} (-1)^{|T|+1} \prod_{i \in T} \mathbf{1}[x_i > t] = \sum_T (-1)^{|T|+1} \mathbf{1}[\text{mín}\, T > t]$ (producto = indicador del mínimo), e integrar en t (o aplicar cualquier funcional lineal). **(5) CP.** $O(2^n)$ en general; útil cuando $\mathbb{E}[\text{mín}\, T]$ tiene forma cerrada (p. ej. $\frac{1}{p(T)}$ para exponenciales o cupones, con DP sobre la suma como arriba). Conviene si el mínimo de un subconjunto es fácil pero el máximo no.

Carrera de exponenciales y Poissonización (1) Contexto. Varios procesos independientes compiten (cola de servidores, cupones con tiempos continuos); se pide quién termina primero o cuánto tarda el último. **(2) Modelo.** $T_i \sim \text{Exp}(\lambda_i)$ independientes, $\Lambda = \sum \lambda_i$. **(3) Resultado.** $\text{mín}_i\, T_i \sim \text{Exp}(\Lambda)$; $\mathbb{P}(T_k = \text{mín}_i\, T_i) = \frac{\lambda_k}{\Lambda}$; el mínimo es independiente de cuál lo alcanza; si $\lambda_i{=}\lambda$: $\mathbb{E}[\text{máx}] = \frac{H_n}{\lambda}$ y $\mathbb{E}[k\text{-ésimo en terminar}] = \frac{1}{\lambda} \sum_{j=0}^{k-1} \frac{1}{n-j}$. En una CTMC, cada estado dura $\text{Exp}(\Lambda_s)$ con $\mathbb{E} = \frac{1}{\Lambda_s}$ y salta a s' con probabilidad $\frac{\lambda_{s \to s'}}{\Lambda_s}$. **(4) Idea.** $\mathbb{P}(\text{mín} > t) = \prod e^{-\lambda_i t} = e^{-\Lambda t}$; falta de memoria $\Rightarrow$ tras cada llegada el resto sigue siendo exponencial con los mismos parámetros. **(5) CP.** $O(1)$ por paso del DP sobre estados; convierte problemas continuos en cadenas discretas ponderadas por $\frac{1}{\Lambda}$. Conviene cuando el enunciado habla de tiempos continuos o tasas.

Ruina del jugador (1) Contexto. Un caminante empieza en i, sube con probabilidad p y baja con $q = 1-p$, y se detiene al llegar a 0 o a N. **(2) Modelo.** Paseo S_n con barreras absorbentes; $r = q/p$; $P_i = \mathbb{P}(\text{llegar a } N \text{ antes que a } 0)$; T = tiempo de parada. **(3) Resultado.** $P_i = \frac{1-r^i}{1-r^N}$ si $p \ne q$ y $P_i = \frac{i}{N}$ si $p = q$; $\mathbb{E}[T_i] = i(N-i)$ si $p = q$, y $\mathbb{E}[T_i] = \frac{1}{q-p}\left(i - N\, \frac{1-r^i}{1-r^N}\right)$ si $p \ne q$. Con una sola barrera (en 0): ruina con probabilidad $\text{mín}(1, r^i)$. **(4) Idea.** Análisis del primer paso: $P_i = pP_{i+1} + qP_{i-1}$, ecuación característica $px^2 - x + q = 0$ con raíces 1 y r; para el tiempo $E_i = 1 + pE_{i+1} + qE_{i-1}$ con solución particular $\frac{i}{q-p}$. Alternativa: r^{S_n} y $S_n - (p-q)n$ son martingalas; teorema de parada opcional. **(5) CP.** $O(1)$ con potencias modulares (si $r \equiv 1 \bmod M$ usar el caso $p = q$ con cuidado). Conviene en pasos aleatorios 1D con barreras; con pasos no unitarios pasa a ecuaciones lineales (siguiente ítem).

Cadenas de Markov absorbentes (1) Contexto. Un proceso salta entre estados con probabilidades fijas hasta caer en un estado terminal; se pide la probabilidad de terminar en cada uno o el tiempo esperado. **(2) Modelo.** $P = \begin{pmatrix} Q & R \\ 0 & I \end{pmatrix}$ con Q transitorio→transitorio y R transitorio→absorbente. **(3) Resultado.** Matriz fundamental $N = (I-Q)^{-1} = \sum_{k \ge 0} Q^k$ (N_{uv} = visitas esperadas a v desde u); tiempos esperados $t = N\mathbf{1}$; probabilidades de absorción $B = NR$; probabilidad de estar en s tras exactamente T pasos: $e_u^\top P^T$. **(4) Idea.** $Q^k \to 0$ (de cada estado transitorio se escapa con probabilidad positiva), luego la serie geométrica matricial converge y $(I-Q)N = I$; sumar sobre columnas da el número esperado de pasos. **(5) CP.** Eliminación gaussiana $O(s^3)$ (modular: verificar que $I-Q$ sea invertible mod M); matrices tridiagonales (1D) en $O(s)$; horizonte T grande con exponenciación $O(s^3 \log T)$ o Kitamasa/Berlekamp–Massey si hay recurrencia lineal; en árboles/DAG escribir $E_v = a_v E_{\text{padre}} + b_v$ y despejar en $O(n)$ ("incógnita lineal"). Conviene en cualquier "esperanza de pasos hasta terminar" con ciclos.

Paseos aleatorios en grafos (1) Contexto. Una partícula salta a un vecino uniforme; se pide tiempo esperado para llegar a v, para

volver, o para visitar todo. **(2) Modelo.** Grafo conexo no dirigido con m aristas; $h(u,v) = \mathbb{E}[\text{pasos } u \to v]$. **(3) Resultado.** Distribución estacionaria $\pi_v = \frac{\deg v}{2m}$; tiempo de retorno $\mathbb{E}_v[T_v^+] = \frac{2m}{\deg v}$; tiempo de conmutación $h(u,v) + h(v,u) = 2m\, R_{\text{ef}}(u,v)$; tiempo de cobertura $\le 2m(n-1)$; en un camino de n nodos con reflexión en 0, $h(0 \to n) = n^2$. **(4) Idea.** π satisface el balance detallado $\pi_u/\deg u = \pi_v/\deg v$ y la esperanza del retorno es $1/\pi_v$ (teorema de renovación); la fórmula de conmutación sale de la analogía con redes eléctricas (leyes de Kirchhoff). **(5) CP.** $O(n^3)$ resolviendo el sistema $h(u,v) = 1 + \frac{1}{\deg u}\sum_{w\sim u} h(w,v)$, o $O(n)$ en árboles. Conviene en problemas de "caminata aleatoria sobre un grafo/árbol".

Reflexión y teorema del escrutinio (1) Contexto. Dos candidatos reciben $a > b$ votos en orden aleatorio; probabilidad de que el ganador vaya siempre adelante (o nunca atrás); conteo de caminos que no tocan una barrera. **(2) Modelo.** Todas las $\binom{a+b}{a}$ secuencias equiprobables. **(3) Resultado.** $\mathbb{P}(\text{A siempre estrictamente adelante}) = \frac{a-b}{a+b}$; $\mathbb{P}(\text{A nunca atrás}) = \frac{a-b+1}{a+1}$; en particular, para $a{=}b{=}n$ la probabilidad de no pasar bajo cero es $\frac{1}{n+1}$ (números de Catalan). **(4) Idea.** Reflexión: los caminos de A a B que tocan la recta $y=-1$ se biyectan con los caminos de A' (reflejado) a B, y $\binom{a+b}{a} - \binom{a+b}{a+1}$ cuenta los buenos; alternativa por rotación cíclica (lema de Dvoretzky–Motzkin). **(5) CP.** $O(1)$ con factoriales precalculados. Conviene en caminos con restricción de signo o de barrera, y para depurar un DP $O(n^2)$ que cuenta lo mismo.

Linealidad de la esperanza en grafos aleatorios (1) Contexto. Esperanza del número de componentes, aristas de un árbol aleatorio, o grados, cuando el grafo se subsamplea al azar. **(2) Modelo.** Indicadores $\mathbf{1}_e$, $\mathbf{1}_v$ sin necesidad de independencia entre ellos. **(3) Resultado.** Bosque con n vértices, aristas conservadas con prob. p: $\mathbb{E}[\#\text{comp}] = n - (n-1)p$; vértices conservados con prob. p: $\mathbb{E}[\#\text{comp}] = np - (n-1)p^2$ (componentes $= V' - E'$ en un bosque). $G(n,p)$: $c(n) = 1 - \sum_{k=1}^{n-1}\binom{n-1}{k-1}c(k)(1-p)^{k(n-k)}$ es $\mathbb{P}(\text{conexo})$ y $\mathbb{E}[\#\text{comp}] = \sum_k \binom{n}{k}c(k)(1-p)^{k(n-k)}$. Árbol de expansión uniforme: $\tau(G) =$ det de cualquier cofactor del Laplaciano, y $\mathbb{P}(e \in T) = R_{\text{ef}}(e)$. Árbol etiquetado uniforme (n^{n-2}, Cayley): $\deg v - 1 \sim \text{Bin}(n-2, \frac{1}{n})$, $\mathbb{E}[\#\text{hojas}] = n(1-\frac{1}{n})^{n-2} \approx \frac{n}{e}$. **(4) Idea.** Para $c(n)$: fijar el componente del vértice 1 (tamaño k, conexo, sin aristas hacia el resto); linealidad $\mathbb{E}[\sum \mathbf{1}] = \sum \mathbb{P}$; Kirchhoff y propiedad de Prüfer. **(5) CP.** $O(n^2)$ para $c(n)$, $O(n^3)$ para el determinante, $O(n)$ para bosques. Conviene cuando el enunciado pide "esperanza de cuántos..." (siempre intentar descomponer en indicadores antes de un DP).

Linealidad en permutaciones aleatorias (1) Contexto. Permutación uniforme de n elementos; se piden puntos fijos, ciclos, récords o inversiones esperadas. **(2) Modelo.** σ uniforme en S_n. **(3) Resultado.** $\mathbb{E}[\#\text{puntos fijos}] = 1$; $\mathbb{E}[\#\text{ciclos}] = H_n$; $\mathbb{E}[\#\text{récords}] = H_n$ (el i-ésimo es récord con prob. $\frac{1}{i}$, independientes); $\mathbb{E}[\#\text{inversiones}] = \frac{n(n-1)}{4}$; $\mathbb{E}[\text{longitud del ciclo de 1}] = \frac{n+1}{2}$; $\mathbb{P}(\text{sin puntos fijos}) = \frac{D_n}{n!} \to \frac{1}{e}$ con $D_n = n!\sum_{k=0}^{n}\frac{(-1)^k}{k!}$. **(4) Idea.** Indicadores $\mathbf{1}[\sigma(i){=}i]$ con $\mathbb{P} = \frac{1}{n}$; $\mathbf{1}[\text{par } (i,j) \text{ invertido}]$ con prob. $\frac{1}{2}$ por simetría; la longitud del ciclo de 1 es uniforme en $\{1,\dots,n\}$; inclusión–exclusión para D_n. **(5) CP.** $O(1)$ o $O(n)$ para sumar. Conviene siempre que haya simetría entre elementos.

Rachas, patrones y fórmula de Conway (1) Contexto. Esperanza de lanzamientos hasta ver k éxitos seguidos, o hasta ver un patrón s (HHT, ABRACADABRA); probabilidad de que un patrón aparezca antes que otro. **(2) Modelo.** Ensayos Bernoulli(p) (o letras iid), T_s = primer tiempo en que s aparece. **(3) Resultado.** Racha de k éxitos: $\mathbb{E}[T] = \sum_{j=1}^{k} p^{-j} = \frac{1-p^k}{(1-p)p^k}$. Patrón s con letras iid: $\mathbb{E}[T_s] = \sum_{\ell:\ \text{pref}_\ell(s)=\text{suf}_\ell(s)} \frac{1}{\mathbb{P}(\text{pref}_\ell(s))}$ (suma sobre los bordes de s, incluido $\ell{=}|s|$); moneda justa: $\sum 2^\ell$. Penney (moneda justa): con $L(X,Y) = \sum_k 2^{k-1}[\text{los últimos } k \text{ de } X = \text{los primeros } k \text{ de } Y]$, $\mathbb{P}(A \text{ antes que } B) = \frac{L(B,B)-L(B,A)}{(L(A,A)-L(A,B))+(L(B,B)-L(B,A))}$. **(4) Idea.** Racha: $E_k = E_{k-1} + 1 + (1-p)E_k$ (si falla, se reinicia). Patrón: casino con apuestas justas, un jugador entra en cada instante y apuesta todo a la siguiente letra; el capital total al detener es $\sum_\ell 1/\mathbb{P}(\text{pref}_\ell)$ para los bordes (martingala y parada opcional). **(5) CP.** $O(k)$ o $O(1)$ con serie geométrica; patrones: bordes con KMP/Z en $O(|s|)$; "primero en aparecer" con dos patrones se resuelve con L en $O(|A|+|B|)$. Conviene en problemas con reinicio condicional (falla ⇒ volver al inicio de un autómata).

Reinicios geométricos e identidad de Wald (1) Contexto. Se repite un intento (que cuesta tiempo aleatorio) hasta que uno tiene éxito; se pide el costo total esperado. **(2) Modelo.** Intentos iid con costo C_i (incluye fallos) y éxito con prob. $q > 0$; N = número de intentos, $T = \sum_{i \le N} C_i$. **(3) Resultado.** $\mathbb{E}[N] = \frac{1}{q}$ y $\mathbb{E}[T] = \mathbb{E}[N]\,\mathbb{E}[C] = \frac{\mathbb{E}[C]}{q}$; en general, para un tiempo de parada τ con $\mathbb{E}[\tau] < \infty$: $\mathbb{E}\big[\sum_{i\le\tau} X_i\big] = \mathbb{E}[\tau]\mathbb{E}[X_1]$. **(4) Idea.** $\sum_{i\le\tau} X_i = \sum_{i\ge 1} X_i \mathbf{1}[\tau \ge i]$ y $\{\tau \ge i\}$ es independiente de X_i (depende solo de $X_1,\dots,X_{i-1}$); tomar esperanza. Contraejemplo: si τ "mira hacia adelante" (p. ej. $\tau =$ último éxito), Wald falla. **(5) CP.** $O(1)$. Conviene en "estrategia con reintentos" y para cerrar DP con autoloop: $E = c + (1-q)E + \cdots \Rightarrow E = \frac{c+\dots}{q}$.

Colisiones y paradoja del cumpleaños (1) Contexto. Cuántos elementos caben en N casillas (hash de 64 bits, códigos, cumpleaños) antes de que haya colisión. **(2) Modelo.** n valores uniformes iid en $\{1,\dots,N\}$. **(3) Resultado.** $\mathbb{P}(\text{sin colisión}) = \prod_{i=0}^{n-1}\left(1 - \frac{i}{N}\right) \le e^{-n(n-1)/(2N)}$; $\mathbb{E}[\#\text{pares colisionados}] = \binom{n}{2}\frac{1}{N}$; $n \approx 1.18\sqrt{N}$ da probabilidad $\frac{1}{2}$; $\mathbb{E}[\text{tamaño hasta 1.ª colisión}] \approx \sqrt{\pi N/2}$. **(4) Idea.** Producto de probabilidades condicionales, $1 - x \le e^{-x}$, y linealidad con indicadores de pares. **(5) CP.** $O(1)$ para estimar error de hashing (p. ej. $n = 10^6$ con $N = 2^{64}$: error $\approx \frac{n^2}{2N} \approx 3 \cdot 10^{-8}$). Conviene al elegir módulos/bases de hashing y al analizar algoritmos aleatorizados.

Test polinomial (Schwartz–Zippel) y verificación aleatoria (1) Contexto. Comprobar si dos expresiones/matrices son iguales (identidad polinómica, $AB = C$, igualdad de multiconjuntos) sin calcularlas por completo. **(2) Modelo.** $P \not\equiv 0$ de grado total d sobre un cuerpo F; r uniforme en S^k, $S \subseteq F$. **(3) Resultado.** $\mathbb{P}(P(r) = 0) \le \frac{d}{|S|}$; Freivalds: si $AB \ne C$ y $x \in \{0,1\}^n$ uniforme, $\mathbb{P}(ABx = Cx) \le \frac{1}{2}$ (error 2^{-t} con t repeticiones). **(4) Idea.** Inducción en el número de variables: un polinomio univariado no nulo de grado d tiene $\le d$ raíces; fijar las demás variables y condicionar. **(5) CP.** Freivalds $O(n^2)$ por prueba frente a $O(n^3)$; hashing polinomial de cadenas con base aleatoria. Conviene para verificar resultados caros con error unilateral controlado.

Problema de la secretaria (parada óptima) (1) Contexto. Se ven n candidatos en orden aleatorio y hay que aceptar o rechazar al instante; se quiere maximizar la probabilidad de elegir el mejor. **(2) Modelo.** Orden uniforme; estrategia: rechazar los primeros $r-1$ y aceptar el primero mejor que todos los anteriores. **(3) Resultado.** $\mathbb{P}(\text{éxito}_r) = \frac{r-1}{n}\sum_{j=r}^{n}\frac{1}{j-1}$ para $r \ge 2$; el óptimo es $r^* \approx \frac{n}{e}$ con probabilidad $\to \frac{1}{e}$. **(4) Idea.** El mejor está en la posición j con prob. $\frac{1}{n}$, y se acepta si el mejor de los $j-1$ anteriores está entre los primeros $r-1$ (prob. $\frac{r-1}{j-1}$); optimizar sobre r. **(5) CP.** $O(n)$ para probar todos los r; para variantes (valores conocidos, costos) se usa DP hacia atrás sobre la "esperanza de seguir". Conviene en parada óptima: $V_i = \max(\text{aceptar}, \mathbb{E}[V_{i+1}])$.

Bayes con hipótesis discretas (1) Contexto. Hay k "tipos" posibles del sistema (dado justo/cargado, jugador fuerte/débil) y se observan resultados. **(2) Modelo.** Prior π_h, verosimilitudes $\ell_h(x)$. **(3) Resultado.** $\mathbb{P}(h \mid x_{1:i}) = \frac{\pi_h \prod_{j\le i}\ell_h(x_j)}{\sum_{h'}\pi_{h'}\prod_{j\le i}\ell_{h'}(x_j)}$ y predicción $\mathbb{P}(x_{i+1} \mid x_{1:i}) = \sum_h \mathbb{P}(h \mid x_{1:i})\,\ell_h(x_{i+1})$. **(4) Idea.** Ley de probabilidad total sobre las hipótesis y regla de Bayes; el denominador es la verosimilitud marginal. **(5) CP.** $O(k)$ por observación; trabajar con logaritmos o renormalizar para evitar underflow, y para respuestas modulares multiplicar por inversos. Conviene cuando el enunciado dice "con probabilidad π_h el sistema es de tipo h" (mezcla discreta, el análogo discreto de Laplace).

Otros patrones que conviene tener a mano Esperanza condicionada: $\mathbb{E}[X] = \mathbb{E}[\mathbb{E}[X \mid Y]]$ para dividir en casos; **varianza total:** $\text{Var}\,X = \mathbb{E}[\text{Var}(X \mid Y)] + \text{Var}(\mathbb{E}[X \mid Y])$. **Funciones generadoras:** $G_X(z) = \mathbb{E}[z^X]$ convierte sumas independientes en productos (convolución con FFT/NTT) y da $\mathbb{E}[X] = G_X'(1)$. **Cotas de cola:** Markov $\mathbb{P}(X \ge a) \le \frac{\mathbb{E}X}{a}$, Chebyshev $\mathbb{P}(|X-\mu| \ge k\sigma) \le \frac{1}{k^2}$, Chernoff para sumas de Bernoulli: $\mathbb{P}(X \ge (1+\delta)\mu) \le e^{-\delta^2\mu/3}$ $(0 < \delta \le 1)$, para justificar algoritmos aleatorizados y heurísticas. **Lema de Borel–Cantelli / método probabilístico:** si $\mathbb{E}[X] = \mu$ existe un objeto con $X \ge \mu$ y otro con $X \le \mu$ (cotas de tipo "siempre existe un corte con $\ge m/2$ aristas"). **Desaleatorización:** método de esperanzas condicionales, escoger cada decisión para no empeorar $\mathbb{E}[X \mid \text{decisiones previas}]$ y obtener un algoritmo determinista.